

IST608 — Database Recovery Techniques

Practice Examination (100 Marks)

Source: *DATABASE RECOVERY (1).pdf* — Elmasri & Navathe, Ch. 22

Coverage: Recovery concepts, WAL, steal/force, deferred/immediate update, checkpoints, rollback, shadow paging, ARIES, multidatabase recovery, catastrophic backup

Mark distribution

Section	Questions	Marks
A — Multiple Choice	20	20
B — Structured	6	30
C — Essay	5	50
Total		100

SECTION A — MULTIPLE CHOICE (20 marks)

Each question carries **1 mark**. Choose the **one** best answer.

1. The main goal of database recovery is to restore the database to:

- (a) The state before any transaction ever ran
- (b) The most recent **consistent** state before the time of failure
- (c) A state with no log file
- (d) Read-only mode permanently

Answer: (b) — Recovery restores the latest consistent state prior to failure.

2. With **deferred update**, physical database changes on disk typically occur:

- (a) Before the transaction starts
- (b) Only after the transaction commits
- (c) Never; only the log is used
- (d) Only during checkpoint, never at commit

Answer: (b) — Deferred update postpones disk updates until commit.

3. For deferred update recovery, which is correct?

- (a) Undo is always required; redo is not
- (b) Undo is not needed; redo may be needed

- (c) Neither undo nor redo is needed
- (d) Only undo-type log entries are required

Answer: (b) — Uncommitted changes are not on disk; committed work may need redo after crash.

4. Immediate update means:

- (a) The database may be updated before the transaction commits, with changes also logged
- (b) No log is ever written
- (c) Commit happens before any log record
- (d) Only read operations are allowed

Answer: (a) — Disk may reflect uncommitted work; logging enables recovery.

5. Undo and redo operations must be:

- (a) Non-idempotent
- (b) Idempotent (multiple executions equivalent to one)
- (c) Performed only on tape backups
- (d) Forbidden under steal/no-force

Answer: (b) — Recovery may repeat operations; idempotency ensures correctness.

6. A dirty bit on a cache buffer indicates:

- (a) The buffer has been modified and may need to be written back
- (b) The transaction has committed
- (c) The page is pinned forever
- (d) Shadow paging is active

Answer: (a) — Dirty = modified in memory; flush to disk if dirty before replacement.

7. A pinned page in the buffer pool:

- (a) Cannot be written back to disk yet
- (b) Is always clean
- (c) Has no dirty bit
- (d) Exists only in shadow directories

Answer: (a) — Pinning prevents premature write-back (e.g., under no-steal).

8. In-place updating means:

- (a) Updated buffer is written to a new disk location only
- (b) Buffer is written to the **same** original disk location, overwriting old values
- (c) No before-image is ever kept
- (d) Shadow directory is updated

Answer: (b) — In-place overwrites the original block location.

9. Write-ahead logging (WAL) requires that before overwriting a data item on disk with its after-image:

- (a) The transaction must abort
- (b) The **before-image (BFIM)** is recorded and the appropriate log entry is flushed to disk
- (c) All transactions are suspended forever
- (d) Only REDO entries exist, never UNDO

Answer: (b) — WAL ensures BFIM is on disk before in-place overwrite for possible UNDO.

10. No-steal means:

- (a) A dirty buffer page updated by a transaction cannot be written to disk before that transaction commits
- (b) All pages are written before any transaction starts
- (c) Stolen pages are recovered from tape only
- (d) The log is never flushed

Answer: (a) — No-steal: uncommitted updates stay in cache until commit.

11. Steal allows:

- (a) Writing updated cache pages to disk before the updating transaction commits
- (b) No logging
- (c) Only deferred update
- (d) No redo after crash

Answer: (a) — Steal permits early write-back of dirty pages (common with steal/no-force).

12. Force means:

- (a) All pages updated by a transaction are written to disk **before** the transaction commits
- (b) No page is ever written until next year
- (c) Only the log is forced, never data pages
- (d) Same as no-force

Answer: (a) — Force: commit waits until all transaction updates are on disk.

13. Most commercial DBMSs typically use:

- (a) Steal / no-force
- (b) No-steal / force only
- (c) No logging
- (d) Shadow paging only in production

Answer: (a) — Steal/no-force reduces I/O and buffer pressure vs force-all-at-commit.

14. A checkpoint (classical) involves:

- (a) Deleting the entire database
- (b) Suspending transactions, force-writing modified buffers, writing checkpoint record, force-writing log
- (c) Only updating statistics, no I/O
- (d) Committing all users' OS sessions

Answer: (b) — Standard checkpoint steps from the lecture.

15. Fuzzy checkpointing differs because:

- (a) Transactions can resume after **begin_checkpoint** is written, before **end_checkpoint**
- (b) No log is used
- (c) It never force-writes buffers
- (d) It replaces ARIES entirely

Answer: (a) — Fuzzy checkpoints reduce downtime; prior checkpoint kept until **end_checkpoint**.

16. Cascading rollback occurs when:

- (a) One transaction rolls back and forces rollback of others that read its uncommitted writes
- (b) Only strict schedules exist
- (c) ARIES analysis phase runs
- (b) Backup is taken to tape

Answer: (a) — Reading uncommitted data from T that aborts can force S to roll back too.

17. Shadow paging recovery after failure:

- (a) Requires full UNDO/REDO from log only
- (b) Discards current directory, frees modified pages — **NO-UNDO/NO-REDO**
- (c) Always uses steal/force
- (d) Cannot work in multiuser systems

Answer: (b) — Revert to shadow directory; old pages remain valid.

18. ARIES recovery phases include:

- (a) Analysis, REDO, UNDO
- (b) Backup only, no log
- (c) Format disk, reload OS
- (d) Two-phase commit only

Answer: (a) — ARIES: analyze → redo (repeat history) → undo (with logging during undo).

19. In multidatabase (distributed) recovery, the coordinator typically uses:

- (a) Shadow paging only

- (b) **Two-phase commit** (prepare, then commit or rollback all)
- (c) No global recovery manager
- (d) Deferred update without log

Answer: (b) — 2PC ensures all sites commit or none do.

20. After **catastrophic failure**, recovery often starts by:

- (a) Ignoring all backups
- (b) Reloading the latest **database backup** and applying log backups since then
- (c) Deleting the log only
- (d) Running UNDO on committed transactions

Answer: (b) — Full backup + incremental log backup limits lost work.

SECTION B — STRUCTURED QUESTIONS (30 marks)

B1. Recovery strategies and failure types (5 marks)

(a) Name two typical recovery strategies from the lecture and when each is best. **(2 marks)**

(b) Distinguish **catastrophic** vs **noncatastrophic** failure in one sentence each. **(2 marks)**

(c) Why may some operations require **redo** after a noncatastrophic crash? **(1 mark)**

Answer

(a)

1. **Restore backed-up copy** of the database — best for **extensive/catastrophic** damage.
2. **Identify inconsistent changes** using the log and undo/redo — best for **noncatastrophic** failure.

(b)

- **Noncatastrophic:** System or transaction failure; log and often recent disk state survive.
- **Catastrophic:** Major damage (fire, wrong tape, widespread disk loss); requires backup + archived log.

(c) Committed transaction updates may be in the log but **not yet written** to disk (no-force, buffering); redo reapplies them.

B2. Steal, force, and typical policies (5 marks)

Complete the table:

Policy	Meaning (one line)
Steal	?
No-steal	?

Policy	Meaning (one line)
Force	?
No-force	?

(b) Which combination do most DBMSs use and why (two reasons)? (2 marks)

Answer

Policy	Meaning
Steal	Dirty page may be written to disk before the updating transaction commits.
No-steal	Dirty page cannot be written until the transaction commits.
Force	All pages updated by T are on disk before T commits.
No-force	Commit allowed before all updated pages are on disk.

(b) **Steal / no-force** — reduces need for very large buffers; reduces disk I/O for heavily updated pages (pages can stay in cache until convenient).

B3. WAL and commit rules (5 marks)

(a) State the **write-ahead logging** rule for UNDO when using in-place update. (2 marks)

(b) What must happen before a transaction's **commit** completes if both UNDO- and REDO-type log records exist? (2 marks)

(c) Define **before-image** and **after-image**. (1 mark)

Answer

(a) The **before-image (BFIM)** must be recorded and the appropriate log entry **flushed to disk** before the after-image overwrites the data item on disk.

(b) Commit cannot complete until **all REDO-type and UNDO-type log records** for that transaction have been **force-written** to disk.

(c) **Before-image**: old value of the item. **After-image**: new value after the write.

B4. Deferred vs immediate update (5 marks)

Aspect	Deferred (NO-UNDO/REDO)	Immediate (UNDO/REDO)
When disk updated	?	?
Undo needed?	?	?
Redo needed?	?	?
Typical steal/force pairing (lecture)	?	steal/no-force for UNDO/REDO

Fill the table. **(4 marks)**

(b) Why is deferred update limited to short transactions? **(1 mark)**

Answer

Aspect	Deferred	Immediate
When disk updated	After commit	Before commit possible
Undo needed?	No	Yes (UNDO-type log)
Redo needed?	Yes (REDO-type log)	Yes (UNDO/REDO or UNDO/NO-REDO)
Steal/force	No-steal (buffers pinned until commit)	Steal/no-force (UNDO/REDO) or steal/force (UNDO/NO-REDO)

(b) Buffers must stay pinned and hold all changes until commit — **buffer space** becomes a problem for long or large transactions.

B5. Checkpoint and fuzzy checkpoint **(5 marks)**

(a) List four steps of taking a **classical checkpoint**. **(2 marks)**

(b) How does **fuzzy** checkpointing reduce disruption? **(2 marks)**

(c) Why are checkpoints useful for recovery time? **(1 mark)**

Answer

(a)

1. Suspend all transactions temporarily
2. Force-write all modified main-memory buffers to disk
3. Write checkpoint record to log; force-write log to disk
4. Resume executing transactions

(b) Processing can **resume after** **begin_checkpoint**; previous checkpoint record kept until **end_checkpoint** — avoids long full quiesce.

(c) Recovery scans **less log** (start redo/undo from last checkpoint rather than from beginning of log).

B6. SQL / application tie-in — rollback and reports **(5 marks)**

(a) A transaction runs **UPDATE Account ...** then crashes before **COMMIT**. What recovery action applies? **(1 mark)**

(b) Why should a report printing “account balance” run **after** commit (or as a post-commit batch job)? **(2 marks)**

(c) Write two SQL lines: start a transaction context (as your DBMS allows), update one row, then **rollback** instead of commit. **(2 marks)**

Answer

(a) Rollback / UNDO — restore old values using undo-type log entries (or discard uncommitted work under deferred update on disk).

(b) If the transaction **fails before commit**, the user must not see a report based on **uncommitted or rolled-back** data; reports should reflect **committed** state only.

(c) Example (syntax varies by DBMS):

```
BEGIN TRANSACTION;    -- or implicit start on first statement
UPDATE Account SET balance = balance - 100 WHERE acc_id = 1;
ROLLBACK;
```

(Accept **START TRANSACTION** / single-statement atomicity discussion if student notes implicit transactions.)

SECTION C — ESSAY QUESTIONS (50 marks)

C1. Recovery concepts: cache, in-place update, and WAL (10 marks)

Explain the recovery process goal, role of the **system log**, **DBMS cache** (dirty bit, pin-unpin), **in-place updating** vs **shadowing**, and **write-ahead logging**. Why must undo/redo be **idempotent**?

Model answer

Goal: Restore the DB to the **most recent consistent state** before failure, preserving **atomicity** (and durability) of transactions.

Log: Records transaction actions (especially writes with old/new values). Survives many failures if on disk; drives undo (backward, restore old values) and redo (forward, reapply committed changes).

Cache: In-memory buffers of disk blocks; **cache directory** tracks what is buffered. **Dirty bit** = modified; must write back before flush if dirty. **Pin** = block cannot be evicted to disk yet (supports no-steal / deferred protocols).

In-place: Write buffer to **same** disk address — overwrites old data; needs WAL with **before-image**.

Shadowing: Write new version elsewhere; directory points to new block; old block kept — multiple versions; **not typical** in commercial systems.

WAL: Before after-image overwrites disk, **BFIM** and log record must be on disk so crash recovery can **UNDO**. Commit may require force-writing all relevant log records first.

Idempotency: Recovery may **repeat** undo/redo steps after partial recovery or nested failures; applying an operation twice must equal applying it once — entire recovery process should be idempotent.

C2. NO-UNDO/REDO (deferred update) with checkpoints (10 marks)

Describe the **deferred update** protocol: when disk is updated, pinning, log type, and commit rules. Explain recovery after crash with and without a **checkpoint** (which transactions need redo). Why is undo unnecessary?

Model answer

Concept: Postpone **physical** database updates until transaction **commits**. Only **REDO-type** log entries needed; **no UNDO-type** entries required for recovery (on disk, uncommitted work was never applied).

Protocol:

- Transaction **cannot** change database on disk until commit.
- All buffers changed by T are **pinned (no-steal)** until commit.
- Commit only after all **REDO** log entries are in log and log buffer **force-written**.

Recovery:

- **No undo** — uncommitted transactions never reached disk (under protocol).
- **Redo** committed transactions whose updates are in log but not yet on disk (e.g., crash after commit log but before page write — though with strict deferred, pages often written at commit).

Checkpoint: Force-write buffers, checkpoint record in log. After crash, recovery starts from **last checkpoint**: redo log forward for transactions that **committed after** checkpoint but whose effects are not fully on disk; ignore or skip work already reflected on disk per checkpoint timeline (Figure 22.2 style reasoning).

Why no undo: Disk never held uncommitted updates; nothing to reverse on disk for incomplete transactions — only need to **complete** committed work (redo).

Limitation: Long transactions holding many pinned dirty buffers exhaust **memory**.

C3. Immediate update: UNDO/REDO and steal/no-force (10 marks)

Contrast **immediate update** with deferred update. Explain **UNDO/REDO** vs **UNDO/NO-REDO** and link to **steal/force** policies. Describe when **UNDO-type** log entries are required and how WAL supports rollback before commit.

Model answer

Immediate update: Database may be updated **before** commit; not every operation must be immediate, but early write-back is allowed. Requires **UNDO-type** log entries (before-images) for active transactions that fail.

UNDO/REDO (steal/no-force): Most common — steal allows dirty pages to disk before commit; no-force allows commit before all pages on disk. On crash: **undo** uncommitted transactions (restore BFIM); **redo** committed transactions whose changes are only in log or buffer.

UNDO/NO-REDO (steal/force): Force writes all updated pages before commit — committed work is on disk; only **undo** needed for losers; no redo for committed T.

Rollback before commit: Transaction fails after update — **undo** log entries restore old values; must avoid **cascading rollback** (other transactions reading uncommitted data) — strict/cascadeless schedules and concurrency control help.

WAL: Ensures BFIM on disk before overwrite so undo is possible after in-place update.

C4. Shadow paging and ARIES (10 marks)

(a) Explain shadow paging: directory, shadow directory, commit, and failure recovery. (b) Explain ARIES: steal/no-force, analysis, redo ("repeating history"), undo, LSN, and why changes are logged during undo.

Model answer

(a) Shadow paging:

- Disk = fixed pages; **directory** maps logical page → disk block.
- On transaction start: copy directory to **shadow directory** (unchanged); **current directory** used for updates.
- On write: **new copy** of page at new location; current directory points to new block; shadow still points to old.
- **Commit:** shadow directory replaced / made current.
- **Failure:** discard current directory, free new pages, revert to shadow — **NO-UNDO/NO-REDO**.
- Multiuser: may still need log for **concurrency control**.

(b) ARIES:

- **Steal/no-force + WAL.**
 - **Analysis:** Find **dirty pages** in buffer and **active transactions** at crash; determine where to start **REDO**.
 - **REDO:** Reapply logged updates from that point — **repeat history** to reconstruct state at crash instant; only **necessary** redos.
 - **UNDO:** Scan log **backward**; undo active transactions in **reverse** order.
 - **LSN (log sequence number):** Address/order of log record; ties change to transaction/page.
 - **Logging during undo:** Prevents repeating completed undo if failure happens **during recovery** (recovery idempotency / correctness).
-

C5. Multidatabase recovery, catastrophic backup, and SQL practice (10 marks)

Explain **two-phase commit** in multidatabase systems (phases, failure in phase 1 vs 2). Describe **database backup** and **frequent log backup** for catastrophes. Outline a recovery scenario: crash after **COMMIT** but before log force-write — which ACID property is at risk and how WAL/commit rules address it.

Model answer

Multidatabase / distributed:

- **Global recovery manager (coordinator) + two-phase commit.**
- **Phase 1 — Prepare:** Coordinator asks participants "ready to commit?" — yes/no.
- **Phase 2 — Commit:** If all yes, coordinator sends **commit**; all sites commit or **none** do.
- **Failure phase 1: Rollback** all.
- **Failure phase 2:** Successful sites can **recover and commit** when coordinator restarts.
- Outcome always recoverable to **all committed** or **all rolled back**.

Catastrophic backup:

- Periodic full **database + log** copy to cheap/off-site storage.
- **Log backed up more frequently** than full DB (smaller) — restores more transactions since last full backup.
- Reload latest backup + apply archived logs after fire, theft, wrong tape, etc.

SQL / ACID scenario:

- **COMMIT** issued but log not **force-written** → crash loses commit record → durability violated; committed work might be lost or ambiguous.
- **Force-write log before commit completes** (and WAL before page overwrite) ensures **durability** and recoverability.
- Application: always end with explicit **COMMIT** or **ROLLBACK**; do not rely on uncommitted reads in reports.

Example recovery narrative: After noncatastrophic crash, ARIES analysis finds T3 active; redo from checkpoint LSN; undo T3 using log; replay committed T1/T2 if pages missing — database returns to consistent state matching serializable committed history.

MARKING GUIDE (INSTRUCTOR)

Section	Total	Notes
A — MCQ	20	1 mark each
B1–B6	30	Partial credit for tables and SQL syntax variants
C1–C5	10 each	Definitions (3–4), mechanisms (3–4), examples/link to ACID/2PC (2–3)

Cross-reference: Pair with *Transaction Processing I* exam for ACID, log record types, strict schedules, and SQL isolation.

End of examination paper.